FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

ADAPTIVE SUDOKU SOLVER

USING CONSTRAINT PROPAGATION, MRV-BASED BACKTRACKING AND MIN-CONFLICTS LOCAL SEARCH

A PROJECT REPORT

Submitted by

BISHAL SUNAR

240701628



Department of Computer Science and Engineering

Rajalakshmi Engineering College

Academic Year 2025–2026

BONAFIDE CERTIFICATE

Certified that this project report "**Adaptive Sudoku Solver Using Constraint Propagation, MRV-Based Backtracking and Min-Conflicts Local Search**" is the bonafide work of "**Bishal Sunar (240701628)**" of the Department of Computer Science and Engineering, Rajalakshmi Engineering College, who carried out the project work for the subject **Foundations of Artificial Intelligence** during the academic year 2025-2026 under my supervision.

Guide: Muneeshwari P

SIGNATURE OF THE FACULTY INCHARGE

Muneeshwari P

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO. / SECTION	PAGE NO.
ABSTRACT	1
CHAPTER 1 : INTRODUCTION	2
CHAPTER 2 : LITERATURE REVIEW	4
CHAPTER 3 : PROPOSED SYSTEM	6
3.6 Architecture Diagram	8
CHAPTER 4 : MODULES DESCRIPTION	9
4.1 Sudoku Input and Validation	9
4.2 Puzzle Analysis and Computational Profiling	9
4.3 Adaptive Solving Engine	10
4.4 Solution Verification and Performance Evaluation	11
CHAPTER 5 : IMPLEMENTATION AND RESULTS	12
5.1 Experimental Setup	12
5.2 Results	14
CHAPTER 6 CONCLUSION AND FUTURE WORK	17
CHAPTER 7 REFERENCES	19
APPENDIX	20

ABSTRACT

Sudoku is a constraint-based puzzle in which digits must be assigned to a grid while satisfying row, column, and 3×3 subgrid constraints. Puzzle difficulty varies, so a single fixed technique may not be ideal for every input. This project presents an Adaptive Sudoku Solver that analyzes puzzle structure and selects or switches among complementary algorithms.

The implementation combines AC-3 arc consistency with naked and hidden singles, CSP backtracking using Minimum Remaining Values (MRV), and Min-Conflicts local search based on heuristic repair swaps within 3×3 boxes. FeatureExtractor and PuzzleAnalyzer compute clue density, candidate variance, an estimated search-space cardinality $\Sigma \log_{10} |D(X_i)|$, and propagation reducibility. The adaptive controller can choose direct propagation, a bounded local-search attempt, or propagation followed by MRV backtracking.

In the reported seven-puzzle benchmark, the Adaptive Solver solved all seven puzzles and recorded an average solving time of 0.00865 seconds. Based on the aggregate values reported in this study, its average runtime was approximately 5.59× as fast than that of naive backtracking, and its average search-node count was approximately 99.5% lower. These comparisons are specific to the tested dataset and implementation. The raw per-puzzle logs should be checked to confirm the averaging method, treatment of unsuccessful runs, and consistency of metric definitions.

CHAPTER 1

INTRODUCTION

1.1 Introduction to Artificial Intelligence

Artificial Intelligence (AI) studies the design of systems that can reason, learn, make decisions, and act to achieve goals. Search and constraint satisfaction are fundamental AI problem-solving approaches. Sudoku is a useful example because its rules are precise, its state space can be represented computationally, and the effectiveness of different algorithms can be measured.

1.2 Sudoku as a Constraint Satisfaction Problem

A Constraint Satisfaction Problem consists of variables, domains, and constraints. In a standard 9×9 Sudoku, each cell is a variable, its domain is a subset of digits 1–9, and the constraints require each digit to appear exactly once in every row, column, and 3×3 subgrid. A solution is a complete assignment that satisfies all constraints.

1.3 Problem Statement

A single fixed algorithm may not be equally efficient for every Sudoku puzzle. Easy puzzles may be solved through forced assignments, while puzzles with more ambiguous candidate choices may require systematic search. Local search may be useful in some cases but can fail to converge within a limited budget. The project therefore investigates a solver that analyzes puzzle features, chooses among multiple established strategies, monitors progress, and changes strategy when necessary.

1.4 Objectives

- Model standard Sudoku as a Constraint Satisfaction Problem.
- Implement constraint propagation, MRV-based backtracking, and Min-Conflicts local search.
- Extract puzzle features such as empty-cell count, candidate-domain sizes, and forced moves.

- Design a feature-based adaptive strategy selector and a progress-monitoring mechanism.
- Verify solutions and compare solver performance on a common benchmark set.
- Measure solving time, success rate, search nodes, backtracks, conflicts where available, and strategy switches.

1.5 Scope

The current evaluation focuses on standard 9×9 Sudoku puzzles represented as a digital grid. The report describes manual grid input and validation as the core input path. Image-based grid detection and digit recognition are considered an extension unless implemented and tested separately. The project combines established AI techniques rather than claiming a new underlying search algorithm.

1.6 Significance

The project demonstrates how a problem analyzer and controller can coordinate different algorithms. It also makes the trade-offs visible: a simple method may be very fast on some inputs but incomplete as a standalone solving process, while a robust search method may solve all benchmark instances with less control overhead than an adaptive framework.

CHAPTER 2
LITERATURE REVIEW

2.1 Constraint Satisfaction and Backtracking

CSP methods represent a problem through variables, domains, and constraints. Backtracking assigns values incrementally and reverses an assignment when it leads to a contradiction. It is a systematic method and, when implemented with suitable checks, can find a solution whenever one exists in a finite CSP. Its practical performance depends strongly on variable ordering and constraint propagation.

2.2 Constraint Propagation

Constraint propagation removes candidate values that conflict with current assignments. In Sudoku, a cell's candidates are the digits not already present in its row, column, or 3×3 subgrid. If only one candidate remains, the value is forced. Propagation can solve puzzles that expose enough forced moves, but propagation alone may stop when no further single-candidate assignment is available.

2.3 Minimum Remaining Values Heuristic

The Minimum Remaining Values (MRV) heuristic selects the unfilled variable with the smallest remaining domain. Choosing the most constrained cell can expose contradictions early and reduce unnecessary branching. Combined with backtracking and forward checking or repeated candidate updates, MRV is a practical strategy for Sudoku.

2.4 Min-Conflicts Local Search

Min-Conflicts is a local-search method for constraint problems. It begins with a candidate assignment, selects a variable involved in a conflict, and changes its value to reduce the number of violated constraints. Unlike systematic backtracking, local search does not inherently guarantee that a solution will be found; performance depends on initialization, repair rules, restart policy, and search budget.

2.5 Adaptive and Hybrid Solving

A hybrid solver can combine methods with different strengths. A feature-based adaptive controller estimates the puzzle's structure and chooses an initial strategy. Progress monitoring can then trigger a switch if the selected strategy stalls or reaches a search limit. Such a design introduces control overhead, so its value must be established through experiments rather than assumed.

2.6 Research Gap

The project focuses on the practical question of whether lightweight puzzle characteristics can guide strategy selection. Rather than treating algorithm comparison as the final goal, it integrates the algorithms in one framework and records the strategy chosen and any switches. The benchmark evaluation examines both successful solving and computational cost.

2.7 Summary

The literature motivates a complementary set of methods: propagation for direct deductions, MRV + backtracking for systematic search, and Min-Conflicts as a local-search alternative. The adaptive controller coordinates these methods and is evaluated against each fixed strategy.

CHAPTER 3

PROPOSED SYSTEM

3.1 System Overview

The proposed Adaptive Sudoku Solver follows an Analyze → Select → Solve → Measure workflow. It receives a 9×9 grid, validates the given values, builds candidate domains, estimates puzzle characteristics, selects a strategy, and attempts to solve the puzzle. The resulting grid is checked independently against Sudoku constraints, and performance metrics are recorded.

3.2 Working Principle

1. Input: receive a 9×9 matrix where 1–9 represent fixed digits and 0 represents an empty cell.
2. Validation: check dimensions, allowed values, and duplicate non-zero digits in each row, column, and subgrid.
3. Feature extraction: compute empty-cell count, candidate sets, candidate distribution, and the number of forced moves.
4. Strategy selection: choose constraint propagation when direct deductions are promising; choose MRV + backtracking when branching is required; consider Min-Conflicts as a bounded local-search alternative.
5. Progress monitoring: track assignments, candidate reduction, search activity, and stagnation or budget exhaustion.
6. Strategy switching: when progress is insufficient, move to another suitable method while preserving a valid state or restarting the method as required by its representation.
7. Verification and reporting: confirm the completed grid and display the selected strategy and measured metrics.

3.3 Strategy Selection Logic

Observed puzzle characteristic	Preferred initial strategy	Reason
Many cells have a single candidate / many forced moves	Constraint propagation	Makes direct deductions without branching.
No sufficient forced moves; constrained cells remain	MRV + backtracking	Systematically explores choices and backtracks on contradictions.
A candidate assignment can be repaired and a bounded local-search attempt is appropriate	Min-Conflicts	Attempts to reduce conflicts through local changes; may require a restart or fallback.
Selected method reaches a budget or makes insufficient progress	Switch to a fallback strategy	Prevents one unsuccessful method from consuming the entire solving effort.

The thresholds used by a real implementation should be stated explicitly and tuned against benchmark data. The report does not claim that empty-cell count alone is a reliable measure of difficulty; candidate structure and observed progress are also relevant.

3.4 Advantages

- Combines complementary AI techniques in one framework.
- Uses measurable features rather than applying one fixed strategy blindly.
- Verifies the final grid independently.
- Supports comparison using repeatable metrics.
- Allows additional algorithms or learned selection policies to be added later.

3.5 Limitations

- Adaptive analysis and switching introduce overhead.
- Puzzle features do not perfectly predict runtime for every puzzle.
- Min-Conflicts may not converge under a limited search budget.
- Results are based on a small benchmark of seven puzzles and should not be generalized without a larger test set.
- Image recognition is not included in the reported benchmark results.

3.6 Architecture Diagram

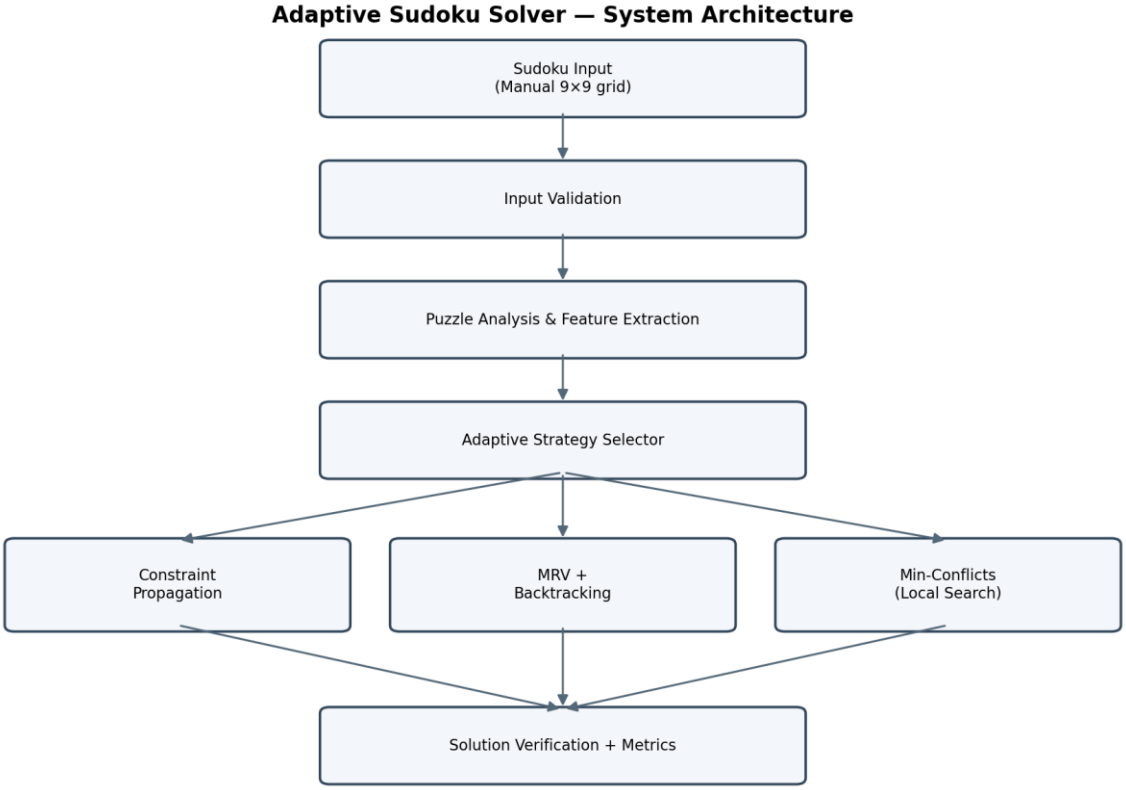


Figure 3.1: Architecture of the Adaptive Sudoku Solver

The architecture separates input handling, analysis, strategy selection, solving, and evaluation. This modular design allows individual solvers to be improved without changing the input-validation or result-verification stages. The diagram should match the implemented program flow; any proposed but unimplemented functions should be labelled as future work.

CHAPTER 4

MODULES DESCRIPTION

4.1 Module 1 — Sudoku Input and Validation

Purpose: accept a standard 9×9 puzzle and reject malformed or contradictory givens before solving.

Input: a 9×9 matrix containing digits 1–9 and zeros for empty cells. Processing checks the row and column count, allowed value range, and duplicate non-zero digits in each row, column, and 3×3 subgrid. Output: a validated grid or a clear invalid-input result. Validation is important because a solver should not waste time searching a puzzle that already violates the rules.

4.2 Module 2 — Puzzle Analysis and Computational Profiling

Purpose: extract features that describe the puzzle's current search structure. For every empty cell, the module computes candidate values by excluding digits already present in the corresponding row, column, and box. It counts empty cells and forced moves and summarizes candidate-domain sizes. These values form a computational profile for the strategy selector.

Feature	Description	Use in selection
Empty cells	Number of unfilled cells	Coarse indication of remaining work.
Candidate domains	Legal digits for each empty cell	Shows how constrained each variable is.
Candidate distribution	Distribution of domain sizes	Helps estimate branching behavior.
Forced moves	Cells with exactly one candidate	Identifies opportunities for propagation.
Progress indicators	Assignments, reductions, search activity	Helps detect whether a strategy is making progress.

4.3 Module 3 — Adaptive Solving Engine

4.3.1 Constraint Propagation

The solver repeatedly computes candidates and assigns values when a cell has a single legal candidate. It continues until the grid is solved, a contradiction is detected, or no further forced assignment is available. This method is efficient when the puzzle exposes enough direct deductions, but it is not sufficient for every Sudoku.

4.3.2 MRV + Backtracking

When propagation cannot complete the puzzle, the solver selects an empty cell with the fewest candidates. It tries each candidate, checks constraints, and recursively solves the remaining grid. If a choice leads to a contradiction, the assignment is undone and another candidate is tried. This provides systematic search and was the strongest fixed strategy in the reported benchmark.

4.3.3 Min-Conflicts Local Search

Min-Conflicts begins with a candidate assignment and iteratively changes a conflicted variable to reduce violations. It is run with a finite search budget. If it fails to make sufficient progress, the adaptive controller can switch to another method. The Min-Conflicts solver solved 3 of the 7 benchmark puzzles (43%). Its performance depended on the initial assignment, repair strategy, restart behaviour, and configured iteration budget.

4.3.4 Adaptive Controller and Progress Monitoring

The controller selects an initial method from the puzzle profile and monitors progress during solving. Indicators may include the number of filled cells, candidate reduction, nodes explored, conflicts, and time or iteration limits. If progress is insufficient, the controller switches to a fallback strategy. The number of strategy switches should be recorded so that adaptive behavior can be evaluated; report the count only when it is available in the execution logs.

4.4 Module 4 — Solution Verification and Performance Evaluation

A completed grid is not accepted solely because the solver terminates. Verification checks that each row, column, and 3×3 subgrid contains digits 1–9 without repetition and that all original clues remain unchanged. The module also records elapsed time, success or failure, search nodes, backtracks, conflicts where available, and the number of strategy switches.

Metric	Meaning
Solving time	Elapsed wall-clock time for the configured solver run.
Success rate	Percentage of benchmark puzzles solved correctly.
Search nodes	Number of search states/nodes explored by the solver.
Backtracks	Number of failed assignments or search reversals.
Conflicts	Constraint violations encountered by the local-search method.
Strategy switches	Number of times the adaptive controller changed methods.

4.5 Data Flow

The validated grid is passed to the analyzer. The analyzer passes a feature profile to the controller. The selected solver returns a candidate grid and execution statistics. The verifier checks correctness, and the evaluator aggregates the metrics for the comparison tables and visualizations.

CHAPTER 5
IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

5.1.1 Hardware and Software Configuration

Item	Configuration reported in project
Programming language	Python 3.13.14
Development environment	Visual Studio Code
Libraries	Python standard library, NumPy, Pandas, Matplotlib
Version control	Git; repository maintained using GitHub
Operating environment	Personal computer running a Python environment
Input representation	Digital 9×9 Sudoku matrix; 0 indicates an empty cell

5.1.2 Benchmark Dataset

The evaluation used seven standard 9×9 Sudoku puzzles spanning Easy, Medium, Hard, and Expert difficulty categories. The benchmark dataset included a minimal 17-clue Sudoku puzzle and Arto Inkala's AI Escargot. The five solver configurations were compared based on solution success rate, average solving time, search nodes, backtracking operations, and constraint propagation steps, where applicable.

Dataset property	Reported details
Puzzles tested	7
Difficulty range	Easy, Medium, Hard, Expert
Special instances	Minimal 17-clue puzzle; AI Escargot

5.1.3 Solver Configurations

Five solver configurations were compared: naive backtracking, CSP + MRV, Constraint Propagation, Min-Conflicts, and the Adaptive Solver. Each configuration was evaluated on the same seven-puzzle dataset using solve count, average runtime, search nodes, backtracks, and propagation steps where available.

5.1.4 Experimental Procedure

1. Run each of the four baseline solvers on the same seven puzzles and record the metrics exposed by the implementation.
2. Run the adaptive solver on the same seven puzzles and preserve its strategy-selection and switching logs.
3. Measure elapsed time, search nodes, backtracks, and propagation steps where the corresponding solver reports them.
4. Verify every claimed solution against Sudoku rules.
5. Aggregate the seven runs per solver and compare completion rate, runtime, and search effort. The reported averages should be reproducible from the raw benchmark output.

5.2 Results

The performance of the five solver configurations was evaluated using the same benchmark dataset of seven standard 9×9 Sudoku puzzles. The evaluation considered the number of puzzles solved, average solving time, average search nodes, backtracking operations, and propagation steps where applicable. Table 5.1 presents the comparative results obtained for each solver configuration.

Solver	Puzzles solved	Success rate	Avg. time (s)	Avg. search nodes	Avg. backtracks	Avg. prop. steps
backtracking	6/7	86%	0.04832	9,263.7	9,220.1	0.0
CSP + MRV	7/7	100%	0.06044	762.0	705.4	761.0
Constraint Propagation	4/7	57%	0.00181	1.0	0.0	73.1
Min-Conflicts	3/7	43%	0.11462	0.0	0.0	0.0
Adaptive Solver	7/7	100%	0.00865	47.3	23.0	119.4

Table 5.1: Comparison of solver performance

5.2.1 Result Analysis

Constraint Propagation solved 4 of 7 puzzles (57%). It was very fast on puzzles where AC-3, naked singles, and hidden singles exposed enough deductions, but it did not complete all puzzles that required search.

CSP + MRV solved all 7 puzzles (100%), with a reported average of 762.0 search nodes, 705.4 backtracks, 761.0 propagation steps, and 0.06044 seconds. Compared with naive backtracking, CSP + MRV recorded substantially fewer search nodes in this benchmark, based on the reported averages.

Min-Conflicts solved 3 of 7 puzzles (43%), averaging 0.11462 seconds. It did not converge on every tested puzzle under the configured initialization and iteration budget. This result does not prove that Min-Conflicts is ineffective for Sudoku in general; initialization, repair rules, restarts, and search budget can affect its behavior.

The Adaptive Solver solved all 7 puzzles (100%) with a reported average runtime of 0.00865 seconds, 47.3 search nodes, 23.0 backtracks, and 119.4 propagation steps. The aggregate summary does not provide a mean strategy-switch count. The reported average runtime corresponds to an approximately 5.59× speedup relative to naive backtracking (0.04832 seconds) in this benchmark; this is a dataset-specific result, not a guarantee for all Sudoku instances.

5.2.2 Adaptive Strategy Selection and Switching

Available detail	Reported information	Use in report	Status	Notes
Puzzle set	7 puzzles	Benchmark scope	Provided	Easy through Expert
Special puzzles	Minimal 17-clue; AI Escargot	Dataset examples	Provided	Per-puzzle mapping not supplied
Strategy sequence	Not included in aggregate summary	Describe controller generally	Needs logs	Do not invent puzzle-specific switches
Adaptive success	7/7	Completion rate	Provided	100%

Table 5.2: Adaptive strategy selection and switching

The supplied aggregate benchmark summary does not include per-puzzle strategy sequences or adaptive switch counts. Do not report puzzle-specific selections or switch counts unless they are confirmed by the execution logs.

5.2.3 Discussion

In the seven-puzzle benchmark, CSP + MRV and the Adaptive Solver each solved all seven puzzles. The Adaptive Solver's reported average runtime was 0.00865 seconds, compared with 0.04832 seconds for naive backtracking, corresponding to an approximately 5.59× runtime ratio. Its reported average search-node count was 47.3, compared with 9,263.7 for naive backtracking, a reduction of approximately 99.5% based on these aggregate values. The reported average backtrack count was 23.0 for the Adaptive Solver and 705.4 for CSP + MRV, corresponding to an approximately 96.7% reduction. These comparisons are specific to the tested benchmark and implementation. Because per-puzzle logs and the detailed aggregation method are not included in the summary, the calculations should be verified against the raw output before the results are treated as reproducible.

5.2.4 Limitations of the Evaluation

- The evaluation was conducted using only seven Sudoku puzzles, which limits the generalizability of the findings to other puzzle sets.
- The absence of repeated trials and detailed per-puzzle measurements limits the assessment of runtime variability and consistency.
- The performance of each solver depends on its implementation, configuration, and search budget.
- Min-Conflicts performance may vary depending on initialization, repair policy, restart strategy, and iteration budget.

- Image-based Sudoku recognition was not evaluated as part of the reported experiments.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The Adaptive Sudoku Solver applies AI problem-solving methods to standard 9×9 Sudoku by representing the puzzle as a Constraint Satisfaction Problem. It combines AC-3 arc consistency and single-candidate deductions, MRV-based CSP backtracking, and Min-Conflicts local search under a controller informed by extracted puzzle features and runtime progress. Input validation and independent solution verification help protect the correctness of the reported output.

In the seven-puzzle benchmark, Constraint Propagation solved 4 of 7 puzzles (57%), CSP + MRV solved all 7 (100%), Min-Conflicts solved 3 of 7 (43%), and the Adaptive Solver solved all 7 (100%). The reported Adaptive Solver average runtime was 0.00865 seconds. Based on the reported averages, this corresponds to an approximately 5.59× speedup relative to naive backtracking and an approximately 99.5% reduction in search nodes. These comparisons are limited to the tested dataset and implementation. The raw per-puzzle logs should be checked to confirm the aggregation method and metric definitions.

6.2 Future Work

- Improve selector thresholds using a larger and more diverse benchmark dataset.
- Evaluate and tune AC-3 arc consistency, naked/hidden singles, and additional consistency techniques on a larger benchmark.
- Explore Algorithm X with Dancing Links (DLX) or SAT-based solving as additional strategies.
- Explore a machine-learning model to predict a suitable strategy from clue density, candidate variance, estimated search-space size, and propagation reducibility.
- Improve progress monitoring using branching factor, candidate reduction, stagnation, and conflict trends.

- Add a restart policy and tune initialization and search budgets for Min-Conflicts.
- Support image-based Sudoku recognition after implementing and evaluating grid detection and digit recognition.
- Extend the solver to larger Sudoku variants and provide visualization of candidate domains and search progress.
- Run repeated trials and report per-puzzle measurements, median times, variability, hardware details, and confidence intervals.

CHAPTER 7

REFERENCES

[1] Russell, S. and Norvig, P. (2021). Artificial Intelligence: A Modern Approach. 4th ed. Pearson.

[2] Minton, S., Johnston, M.D., Philips, A.B. and Laird, P. (1992). Minimizing conflicts: A heuristic repair method for constraint satisfaction and scheduling problems. Artificial Intelligence, 58(1–3), pp. 161–205. https://doi.org/10.1016/0004-3702(92)90007-K.

[3] Reeson, C.G., Huang, K.C., Bayer, K.M. and Choueiry, B.Y. (2007). An Interactive Constraint-Based Approach to Sudoku. Proceedings of the AAAI Conference on Artificial Intelligence, Vancouver, Canada, pp. 1976–1977. https://cdn.aaai.org/AAAI/2007/AAAI07-362.pdf

[4] Knuth, D.E. (2000). Dancing Links. arXiv:cs/0011047. https://arxiv.org/abs/cs/0011047

[5] Stuckey, P.J., Feydy, T., Schutt, A., Tack, G. and Fischer, J. (2017). A Hybrid Approach for the Sudoku Problem: Using Constraint Programming in Iterated Local Search. IEEE Intelligent Systems, 32(2), pp. 52–62. https://doi.org/10.1109/MIS.2017.29.

[6] Carnegie Mellon University. Constraint Satisfaction Problems and Min-Conflicts teaching material. https://www.cs.cmu.edu/~ggordon/780-spring09/slides/CSPs%2C%20SAT%2C%20FOL%20resolution%20strategies.pdf

APPENDIX

A. SOURCE CODE

The complete source code of the Adaptive Sudoku Solver is provided with the project materials. The Python implementation includes constraint propagation, CSP-based backtracking with the Minimum Remaining Values (MRV) heuristic, Min-Conflicts local search, puzzle feature extraction, adaptive strategy selection, and solution validation.

The main program, main.py, provides command-line options to solve a Sudoku puzzle, analyze its difficulty, compare different algorithms, and execute benchmarks. The benchmark and analysis scripts are available in the experiments directory.

The main components of the implementation are described below.

1. Constraint Propagation Solver

```python
import pytest
from src.models.sudoku import Sudoku
from src.solvers.constraint_propagation import ConstraintPropagationSolver
from src.utils.validator import is_valid_solution, matches_initial_clues


def test_constraint_propagation_on_easy_puzzle():
    # Puzzle 1
    grid = [
        [5, 3, 0, 0, 7, 0, 0, 0, 0],
        [6, 0, 0, 1, 9, 5, 0, 0, 0],
        [0, 9, 8, 0, 0, 0, 0, 6, 0],
        [8, 0, 0, 0, 6, 0, 0, 0, 3],
        [4, 0, 0, 8, 0, 3, 0, 0, 1],
        [7, 0, 0, 0, 2, 0, 0, 0, 6],
        [0, 6, 0, 0, 0, 0, 2, 8, 0],
        [0, 0, 0, 4, 1, 9, 0, 0, 5],
        [0, 0, 0, 0, 8, 0, 0, 7, 9],
    ]
    initial_copy = [row[:] for row in grid]
    sudoku = Sudoku(grid)
    solver = ConstraintPropagationSolver(sudoku)

    solved = solver.solve()
    assert solved is True
    assert sudoku.is_solved()
    assert is_valid_solution(sudoku.grid)
    assert matches_initial_clues(initial_copy, sudoku.grid)
    assert solver.propagation_steps > 0
    assert solver.backtracks == 0
```

2. CSP Solver with MRV

```python
from src.models.sudoku import Sudoku
from src.solvers.csp_mrv import CSPSolver


def test_csp_mrv_solver():

    grid = [
        [5, 3, 0, 0, 7, 0, 0, 0, 0],
        [6, 0, 0, 1, 9, 5, 0, 0, 0],
        [0, 9, 8, 0, 0, 0, 0, 6, 0],
        [8, 0, 0, 0, 6, 0, 0, 0, 3],
        [4, 0, 0, 8, 0, 3, 0, 0, 1],
        [7, 0, 0, 0, 2, 0, 0, 0, 6],
        [0, 6, 0, 0, 0, 0, 2, 8, 0],
        [0, 0, 0, 4, 1, 9, 0, 0, 5],
        [0, 0, 0, 0, 8, 0, 0, 7, 9]
    ]

    sudoku = Sudoku(grid)
    solver = CSPSolver(sudoku)

    result = solver.solve()

    assert result is True
    assert sudoku.is_solved()
    assert solver.nodes > 0
    assert solver.propagation_steps > 0
```

3. Min-Conflicts solver

```python
from src.models.sudoku import Sudoku
from src.solvers.min_conflicts import MinConflictsSolver

def test_min_conflicts_solver():

    grid = [
        [5, 3, 0, 0, 7, 0, 0, 0, 0],
        [6, 0, 0, 1, 9, 5, 0, 0, 0],
        [0, 9, 8, 0, 0, 0, 0, 6, 0],
        [8, 0, 0, 0, 6, 0, 0, 0, 3],
        [4, 0, 0, 8, 0, 3, 0, 0, 1],
        [7, 0, 0, 0, 2, 0, 0, 0, 6],
        [0, 6, 0, 0, 0, 0, 2, 8, 0],
        [0, 0, 0, 4, 1, 9, 0, 0, 5],
        [0, 0, 0, 0, 8, 0, 0, 7, 9]
    ]

    sudoku = Sudoku(grid)

    solver = MinConflictsSolver(
        sudoku,
        max_steps=50000,
        max_restarts=20
    )

    result = solver.solve()

    assert result is True
    assert sudoku.is_solved()
```

4. Adaptive sudoku solver

```python
class AdaptiveSudokuSolver:
    """
    An intelligent, meta-heuristic Sudoku solver that dynamically adapts its
    strategy based on structural problem features.
    """

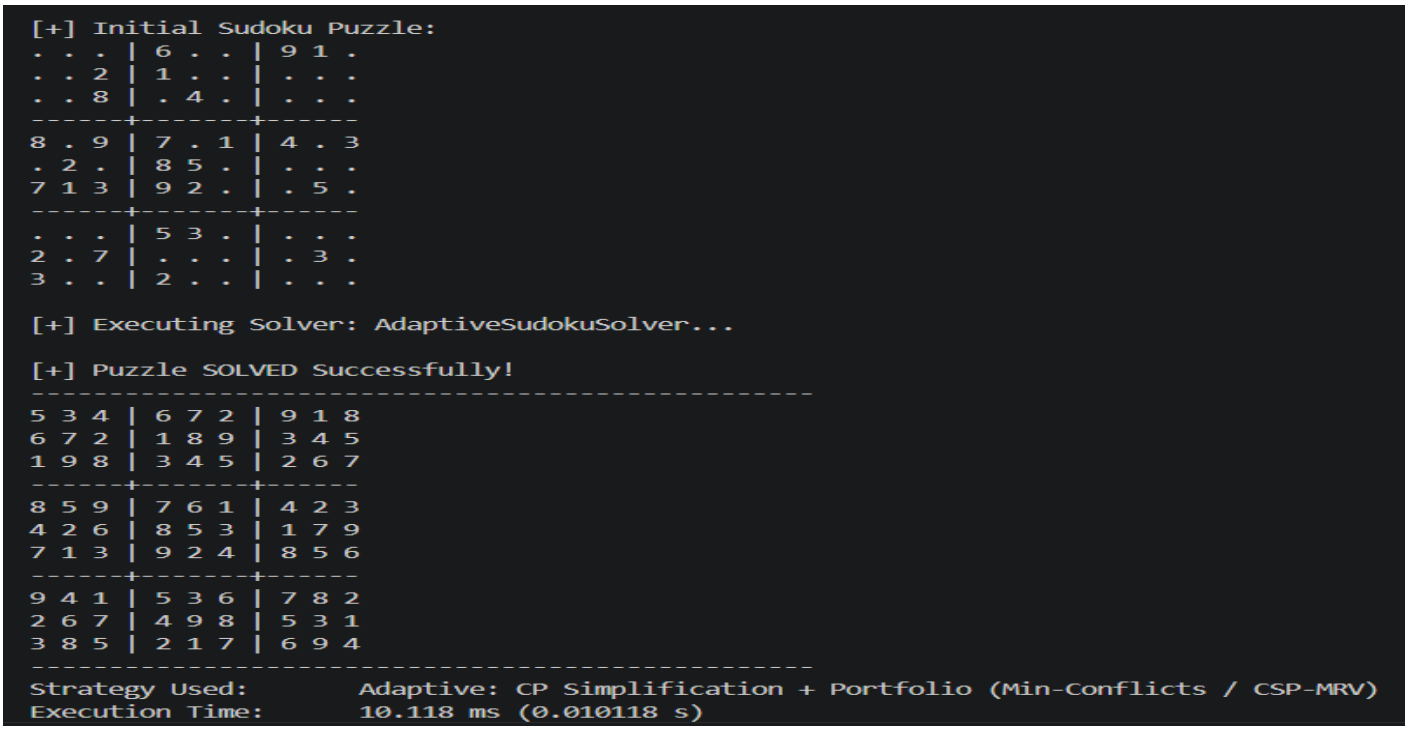
    def __init__(self, sudoku: Sudoku, strategy_mode: str = "auto"):
        """
        Args:
            sudoku: The Sudoku puzzle to solve.
            strategy_mode: 'auto', 'cp', 'mrv', 'min_conflicts', 'backtracking', or 'hybrid'.
        """
        self.sudoku = sudoku
        self.strategy_mode = strategy_mode.lower()

        # Metrics attributes (compatible with SolverMetrics)
        self.nodes = 0
        self.backtracks = 0
        self.propagation_steps = 0
        self.steps = 0
        self.restarts = 0
        self.conflicts = 0

        # Analysis metadata
        self.features: Optional[PuzzleFeatures] = None
        self.analyzer: Optional[PuzzleAnalyzer] = None
        self.difficulty: Optional[DifficultyLevel] = None
        self.difficulty_score: float = 0.0
        self.selected_strategy: str = ""
        self.phases: List[Dict[str, Any]] = []

    def solve(self) -> bool:
        """
        Solve the puzzle using the selected strategy mode.
        """
        if self.strategy_mode == "cp":
            return self._solve_cp()
```

B. Output Screenshot



C. Test Cases

Table C.1 lists the test cases used to evaluate the functionality and correctness of the Adaptive Sudoku Solver. The tests cover puzzle validation, conflict detection, constraint propagation, backtracking, adaptive solving, and solution verification.

Table C.1: Test Cases for the Adaptive Sudoku Solver

Test Case	Expected Behaviour	Observed Result
Valid Sudoku puzzle	The program accepts the puzzle for analysis and solving.	The puzzle was accepted successfully. The validator returned True, and the 9×9 grid containing 30 clues was loaded into the solving pipeline

Puzzle with conflicting initial clues	The validator detects the conflict and rejects the invalid puzzle.	The validator returned False for duplicate digit 5 in row 0. The CLI displayed a warning and aborted solver execution.
Puzzle solvable by constraint propagation	The solver completes the puzzle when deterministic deductions are sufficient.	The solver successfully solved the puzzle in 1.786 ms using 102 propagation steps, exploring 1 node with 0 backtracks.
Puzzle requiring backtracking	CSP + MRV searches for a valid solution when additional search is required.	Constraint propagation stalled at 24/81 assigned cells. CSP + MRV successfully solved the puzzle in 18.837 ms, visiting 218 nodes and performing 159 backtracks.
Puzzle solved by the adaptive approach	The adaptive controller selects a suitable strategy and produces a valid solution.	The puzzle was classified as Hard (Score: 88.3/100). The adaptive solver selected CP simplification followed by CSP-MRV backtracking and solved the puzzle in 19.701 ms, visiting 216 nodes and performing 158 backtracks.
Verification of completed solution	The validator checks that the solution satisfies all Sudoku constraints.	The validator confirmed that the solution was valid and all original clues were preserved. Both is_valid_solution and matches_initial_clues returned True.